

Text-Preprocessing

Regular Expressions (Regex)

Patterns

- * : 0 or more
- + : 1 or more
- ? : 0 or 1
- . : any character
- ^ : start of string
- \$: end of string
- {n} : exactly n times
- {n,} : n or more times
- {n,m} : between n and m times
- {,m} : up to m times
- [abc]' : a or b or c
- '[a-z]': : any character from a to z
- '[abc]': : any character except a or b or c
- \d : digit [0-9]
- \D : non-digit [~0-9]
- \w : word character [a-zA-Z0-9.]
- \W : non-word character [~a-zA-Z0-9.]
- .^ : any character (0 or more times)

- Precedence order:
 1. Parentheses ()
 2. Counters (*, +, ?, {}, etc)
 3. Sequences/anchors (ab, ^, \$)
 4. Disjunction (—)

Tokenization

- Top-down tokenization (rule-based). For languages that use space characters between words.
- Bottom-up tokenization (data-driven approach, e.g. BPE). Subword tokenization. Better at handling out-of-vocab words and rare words.
- Can't blindly remove punctuation symbols (e.g Ph.D, \$45.55, 01/04/2002, URLs)

Penn Treebank Tokenization Standard

- Seperate out clitics (doesn't → does n't)
- Keep hyphenated words together
- Separate out punctuation symbols

Byte-Pair Encoding

1. **Token learner.** Input: Raw training corpus. Output: Vocab (set of tokens)
 - Initialise vocab to set of all unique single characters: {A, B, C, ..., a, b, c}
 - Add special end-of-word symbol ' ' before space in training corpus
 - Note: Make all words character separated. Have a count of the words.
 - Choose 2 symbols that are most frequently adjacent in training corpus (say 'X', 'Y')
 - Add a new merged symbol 'XY' to the vocab
 - Replace every adjacent 'X', 'Y' in the corpus with 'XY'
 - Repeat until *k*-merges have been done
2. **Token segmenter.** Input: Raw test sentence and Vocab. Output: Tokenized sentence.
 - Make all words character separated.
 - Greedily merge characters in the order we learned them.

Normalization

- Lemmatization:** Represent all words as their lemma, their shared root (dictionary headword form). (e.g. am, is, are → be; car, cars, car's, cars' → car)
- Morphemes:**
 - Stems: The core meaning-bearing units
 - Affixes: parts that adhere to stems, often with grammatical function
- Porter Stemming Algorithm:** A series of rewrite rules run in a cascade. Reduces words to stems by chopping off affixes crudely. E.g (ATIONAL → ATE, ING → e if stem contains vowel, SSES → SS)

Min-Edit Distance

- Source: Delete operation. Bottom up. Cost = 1
- Target: Insert operation. Left to right. Cost = 1
- Diagonal: Substitution operation. Diagnoal. Cost = 0 if same characters, 2 otherwise.

N-Gram Language Models

- Probability of a language sequence. Word Prediction.
- Use the previous *n* − 1 words to predict the next word.
$$P(w_n|w_{1:n-1})$$
$$P(w_{1:n}) = P(w_1) \times P(w_2|w_1) \times P(w_3|w_{1:2}) \times \cdots \times P(w_n|w_{1:n-1}) = \prod_{k=1}^n P(w_k|w_{1:k-1})$$
- Estimating n-gram probability:
 - **Maximum Likelihood Estimate (MLE)**
$$P(w_n|w_{1:n-1} = \frac{C(w_{w1:n})}{C(w_{w1:n-1})}$$
 (n-gram)
 - $P(w_2|w_1 = \frac{C(w_1w_2)}{C(w_1)}$ (bigram)
 - Data sparsity when estimating probability of longer n-grams
- N-gram approximation:
 - Markov assumption:
$$P(w_k|w_{1:k-1}) \approx P(w_k|w_{k-(N-1):k-1})$$
 - *w_k* only depends on its previous *N* − 1 words (*N* ≤ *k*)
 - Bigram Approximation:
$$P(w_{1:n}) \approx \prod_{k=1}^n P(w_k|w_{k-1})$$
 - Use logprob instead as multiplying many probabilities results in numerical underflow

Evaluating N-Gram Models

- Extrinsic vs Intrinsic Evaluation

Perplexity

- $PP = m(w_1, \dots, w_n)^{-\frac{1}{n}} = \frac{1}{\sqrt[n]{m(w_1, \dots, w_n)}}$
- The **lower** the perplexity the better
- If perplexity = k, the model is uncertain as if it had to choose uniformly among k equally likely words at each step. Weighted average branching factor of a language.
- Bigram LM:
$$PP = \frac{1}{\sqrt[n]{m(w_1, \dots, w_n)}} = \frac{1}{\sqrt[n]{\prod_{k=1}^n P(w_k|w_{k-1})}}$$

Smoothing

- Lowering non-zero n-gram counts in order to assign some probability mass to zero n-grams
- Smoothed bigram count:
$$P(w|w_0) = \frac{C(w_0w)+1}{C(w_0)+V} = \frac{C^*(w_0w)}{C(w_0)}$$
- Discount $d = \frac{C^*(w_0w)}{C(w_0)}$

- Add-k smoothing for bigram: $P(w|w_0) = \frac{C(w_0w)+k}{C(w_0)+kV}$

Interpolation

- Combining benefits of higher order n-grams with lower-order n-grams
- Bigrams: $\hat{P}(w_0|w_{-1}) = \lambda_1P(w_0|w_{-1}) + \lambda_2P(w_0)$
- Trigrams: $\hat{P}(w_0|w_{-2}w_{-1}) = \lambda_1P(w_0|w_{-2}w_{-1}) + \lambda_2P(w_0|w_{-1}) + \lambda_3P(w_0)$
- Note: $\sum_i \lambda_i = 1$. Select λ_i that maximises the probability of the development set.

Entropy

- $H(X) = - \sum_{x \in X} p(x) \log_2 p(x)$
- Number of bits to encode information in the optimal coding scheme
- Entropy of a sequence in language *L*:
$$H(w_1, w_2, \dots, w_n) = - \sum_{w_{1:n} \in L} p(w_{1:n}) \log_2 p(w_{1:n})$$
- Entopy rate (per-word entropy):
$$\frac{1}{n} H(w_{1:n}) = - \frac{1}{n} \sum_{w_{1:n} \in L} p(w_{1:n}) \log_2 p(w_{1:n})$$
- $H(L) = \lim_{n \rightarrow \infty} - \frac{1}{n} \sum_{w_{1:n} \in L} p(w_{1:n}) \log_2 p(w_{1:n})$
- By Shannon-McMillan-Breiman theorem:
$$H(L) = \lim_{n \rightarrow \infty} - \frac{1}{n} \log_2 p(w_1, \dots, w_n)$$
- Cross Entropy:** Used for comparing two language models, *m* and *p*
$$H(p, m) = \lim_{n \rightarrow \infty} - \frac{1}{n} p(w_{1:n}) \log_2 m(w_{1:n})$$
- By Shannon-McMillan-Breiman theorem:
$$H(p, m) = \lim_{n \rightarrow \infty} - \frac{1}{n} \log_2 m(w_{1:n})$$
- Property: $H(p) \leq H(p, m)$. Difference between them is a measure of how accurate model *m* is.
- More accurate the model, the **lower its cross-entropy**
- Perplexity (PP)** = $2^H = 2^{-\frac{1}{n} \log_2 m(w_{1:n})}$

Paired Bootstrap Test

- Create a test set *x_i* of the same size as *x* by sampling from *x* with replacements
- Calculate $\delta(x_i) = M(A, x_i) - M(B, x_i)$
- $s = s + 1$ if $\delta(x_i) \leq 0$
- p-value = $\frac{s}{b}$. *b* is num of samples
- If p-value < 0.01, then A is better than B at 0.01 level of significance

Deep Learning in NLP

Linear Models

- $y = f(x) = x \cdot W + b$
- $x \in \mathbb{R}^{d_{in}}; W \in \mathbb{R}^{d_{in} \times d_{out}}; y, b \in \mathbb{R}^{d_{out}}$
- In the matrix representation, add a 1 to the *x* input and *b* to the last row of *W*
- Binary classification: Perceptron.
$$class(x) = \hat{y} = 1(\text{if } f(x) \geq 0); 0(\text{if } f(x) < 0)$$
- Sigmoid function:** maps numbers to 0-1. $\sigma(x) = \frac{1}{1+e^{-x}}$
- Logistic regression:**
 - $P(\hat{y} = 1|x) = \sigma(x \cdot w + b)$
 - $P(\hat{y} = 0|x) = 1 - P(\hat{y} = 1|x) = 1 - \sigma(x \cdot w + b)$
 - $class(x) = \hat{y} = 1(\text{if } P(\hat{y} = 1|x) \geq 0); 0(\text{otherwise})$
- Mutli-class Classification:** $class(x) = \hat{y} = \operatorname{argmax}_i \hat{y}_{[i]}$
- Softmax function:** maps output to probability distribution.
$$\operatorname{softmax}(x) = \frac{e^{x_{[i]}}}{\sum_j e^{x_{[j]}}}$$
. Use in multinomial logistic regression.

Training & Loss Functions

- $\hat{\theta} = \operatorname{argmin}_{\theta} (\frac{1}{n} \sum_{i=1}^n L(f(x_i; \theta), y_i) + \lambda R(\theta))$
- Binary Cross-Entropy Loss:**
$$L_{\text{logistic}}(\hat{y}, y) = -y \log_2 \hat{y} - (1 - y) \log_2 (1 - \hat{y})$$
- Categorical Cross-Entropy Loss:**
$$L_{\text{c-e}}(\hat{y}, y) = - \sum_i y_{[i]} \log_2 (\hat{y}_{[i]})$$
- Hard Classification Problem**, one *t* as single correct class:
$$L_{\text{c-e}}(\hat{y}, y) = - \log_2 (\hat{y}_{[t]})$$
- Squared (quadratic) loss:** $L(\hat{y}, y) = \frac{1}{2} (\hat{y} - y)^2$
- Regularization:**
 - $R_{L_2}(W) = ||W||_2^2 = \sum_{i,j} (W_{[i,j]})^2$
 - $R_{L_1}(W) = ||W||_1 = \sum_{i,j} |W_{[i,j]}|$
- Gradient Descent:**
 - $L(\mathbf{w}_1) > L(\mathbf{w}_2) > \cdots > \min$
 - $\delta L(w_0, w_1, \dots, w_{m-1}) = (\frac{\delta L}{\delta w_0}, \frac{\delta L}{\delta w_q}, \dots, \frac{\delta L}{\delta w_{m-1}})$
 - $L(\mathbf{w}, \mathbf{v}) \approx L(\mathbf{w}) + \mathbf{v} \cdot \Delta L(\mathbf{w})$
 - $\mathbf{v} = -\alpha \Delta L(\mathbf{w}); \alpha > 0$
 - $L(\mathbf{w} - \alpha \Delta L(\mathbf{w})) \approx L(\mathbf{w}) - \alpha ||\Delta L(\mathbf{w})||^2$
 - Hence: $L(\mathbf{w}) > L(\mathbf{w} - \alpha \Delta L(\mathbf{w}))$
 - $w_{t+1} = w_t - \alpha_t \Delta L(\mathbf{w}_t)$

Feed-Forward Neural Networks

- XOR Problem.** Can be solved by applying a nonlinear input transformation.
- Multilayer Perceptron (MLP)** with 1 hidden layer:
$$NN_{MLP1}(x) = g(\mathbf{xW}^1 + \mathbf{b}^1)\mathbf{W}^2 + \mathbf{b}^2$$
- Activation Functions:**
 - Sigmoid. Nice property: $\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$
 - Hyperbolic Tangent. $\tanh(x) = \frac{e^{2x}-1}{e^{2x}+1} = \frac{e^x-e^{-x}}{e^x+e^{-x}}$.

Nice property: $\frac{d}{dx} \tanh(x) = 1 - [\tanh(x)]^2$
 - Hard tanh = -1 if *x* < −1; 1 if *x* > 1; x otherwise
 - ReLU = max(0, *x*)
- Dropout Regularization:** Multiply the output of a hidden layer with random masking vectors (setting random neurons to 0). Prevent overfitting.

Neural Network Training

- Multivariate Chain Rule:**
$$\frac{\delta L}{\delta w_i} = \frac{\delta L}{\delta x_1} \frac{\delta x_1}{\delta w_i} + \frac{\delta L}{\delta x_2} \frac{\delta x_2}{\delta w_i} + \cdots + \frac{\delta L}{\delta x_n} \frac{\delta x_n}{\delta w_i}$$
- Backpropagation. Recursive calculation.
- Computational Graph: A directed acyclic graph. Nodes: mathematical operations or bound variables. Edges: flow of intermediary values between nodes

Word Embeddings (Word2Vec)

- Sparse (One-hot) vs Dense vectors (Learned)
- Two vectors for each word: **center**, **context**
- Make the vector **w** of center word close to vectors **c** of words in its context. Binary prediction task
- Training distinguishes *D* and $\bar{D}$
- Negative words sampled: $\frac{\#(w)^{0.75}}{\sum_{w'} \#(w')^{0.75}}$
 1. Treat target word **w** and neighbouring context word **c** as positive examples
 2. Randomly sample other words to get negative examples
 3. Use **logistic regression** to train a classifier to distinguish +ve and -ve examples
 - $P(D = 1|w, c) = \frac{1}{1+e^{-w \cdot c}}$
 - $P(D = 0|w, c) = 1 - P(D = 1|w, c) = \frac{1}{1+e^{w \cdot c}}$

- Minimise: $L(\theta, D, \bar{D}) = -\sum_{(w,c) \in D} \log P(D = 1|w, c) - \sum_{(w,c) \in \bar{D}} \log P(D = 0|w, c)$
- CBOW: $\log P(D = 1|w, c_{1:k}) = \log \frac{1}{1 + e^{-(w \cdot c_1 + w \cdot c_2 + \dots + w \cdot c_k)}}$
- Skipgram: $\log P(D = 1|w, c_{w:k}) = \sum_{i=1}^k \log \frac{1}{1 + e^{-w \cdot c_i}}$
- Gradient Descent. Final embedding: $w_i + c_i$ **or** w_i
- 4. Used the learned weight vectors as embeddings.
- Cosine similarity: $\text{sim}_{cos}(u, v) = \frac{\mathbf{u} \cdot \mathbf{v}}{||\mathbf{u}|| ||\mathbf{v}||}$
- Used for word analogy task via vector algebra

Neural Language Models

- $P(w_i|w_1, w_2, \dots, w_{i-1}) = \frac{P(w_1, w_2, \dots, w_i)}{P(w_1, w_2, \dots, w_{i-1})}$
- $P(w_{1:i}) = P(w_1) \times P(w_2|w_1) \times \dots \times P(w_i|w_{1:i-1})$
- **Downsides of n-gram LMs**: requires smoothing; number of n-grams grow multiplicatively; lack of generalization
- **Input**: a sequece of k words
- **Output**: probability distribution over next word
- $\hat{y} = P(w_i|w_{i:k}) = LM(w_{1:k}) = \text{softmax}((g(\mathbf{x}\mathbf{W}^1 + \mathbf{b}^1))\mathbf{W}^2 + \mathbf{b}^2)$
- Parameters: $\mathbf{E}_{[w]}$, $\mathbf{W}^1$, $\mathbf{b}^1$, $\mathbf{W}^2$, $\mathbf{b}^2$.
Note: $E \in \mathbb{R}^{|V| \times d_w}; x = [v(w_1); v(w_2); \dots; v(w_k)]$
- Training samples: sequence of k words paired with $k + 1^{\text{th}}$ word as target label for classification
- Loss function: Categorical cross-entropy loss

RNN

- Model non-Markovian dependencies. Deep NN.
- $RNN^*(x_{1:n}; s_0) = y_{1:n}$
- $s_i = R(s_{i-1}, x_i); y_i = O(s_i)$
- $x_i \in \mathbb{R}^{d_{in}}, y_i \in \mathbb{R}^{d_{out}}, s_i \in \mathbb{R}^{f(d_{out})}$
- sequence labelling, sequence classification, language modeling, encoder-decoder
- **Sentence Level Classification**: $\hat{y} = \text{softmax}(MLP(RNN(x_{1:n})))$. Cross-entropy loss. Using pre-trained word embeddings.
- **Sequence Labelling**: Produce an output $\hat{t}_i$ after each input x_i is read. Loss: $L(\hat{t}_{1:n}, t_{1:n}) = \sum_{i=1}^n L_{\text{local}}(\hat{t}_i, t_i)$
- **Language Modelling**: output at a timestep acts as input to the next time step, $y_i = x_{i+1}$. Loss = $L = -\sum_t \log_2(\hat{y}_t)$
- Conditioned Generation: Concatenate embedded input vector with a condition vector, $\mathbf{c} \ s_j = R(\mathbf{s}_{j-1}, [\hat{\mathbf{t}}_j; \mathbf{c}])$
- **Encoder-Decoder**: Sequence to sequence
 - Encoder: $RNN^*_{enc}(x_1, \dots, x_n) = c_1, \dots, c_n$
 - Decoder: Conditioned generator RNN.
- $\bar{a}_{[i]}^j = \begin{cases} s_{j-1} \cdot c_i & \text{dot product} \\ s_{j-1} W c_i & \text{bilinear} \\ \tanh([s_{j-1}; c_i]U + b) \cdot v & \text{MLP} \end{cases}$
- $a^j = \text{softmax}(\bar{a}_{[1]}^j, \dots, \bar{a}_{[n]}^j)$
- $\mathbf{c}^j = \sum_{i=1}^n a_{[i]}^j \cdot c_i \leftarrow \text{attend}$
- $s_j = R_{\text{dec}}(s_{j-1}, [\hat{t}_j; \mathbf{c}^j])$
- $y_j = O_{\text{dec}}(s_j)$

Transformers

- Exploit parralel Preprocessing
- Transformer blocks are stacked on top of one another. Hence, input and output dimensions are the same.

- **input** $\rightarrow$ **Self-Att Layer** $\rightarrow$ **Layer Normalise** $\rightarrow$ **Feed-Forward** $\rightarrow$ **Layer Normalise** $\rightarrow$ **output**. Residual connection before each layer normalise.
- Decoder-only, Encoder-only, Encoder-decoder
- Decoder-Only Transformer**
- Self Attention